Ingenieria de Eventide

Este documento describe la arquitectura tecnica de Eventide, su estructura de carpetas, y como extenderlo. Es el informe tecnico de ingenieria del proyecto.

Audiencia: desarrolladores, ingenieros de software, reclutadores tecnicos.

---

1. Arquitectura general

Eventide es un sistema neuro-simbolico multi-agente compuesto por 3 capas:

- Capa de presentacion: interfaz web en Streamlit.
- Capa de servicio: API REST en Flask.
- Capa de agentes: sistema multi-agente con colas de mensajes.
- Capa de datos: modulos core (embedding, optimizador, grafo, simulacion).

Flujo de una consulta:

1. Usuario escribe query en la UI (Streamlit).
2. UI llama a la API REST (POST /api/v1/buscar).
3. API envia mensaje al agente buscador.
4. Buscador ejecuta busqueda semantica con FAISS.
5. API devuelve resultados a la UI.
6. UI llama a POST /api/v1/agenda con los resultados.
7. API envia mensaje al agente optimizador.
8. Optimizador ejecuta Simulated Annealing.
9. API devuelve la agenda optimizada.
10. UI llama a POST /api/v1/narrar con la agenda.
11. API envia mensaje al agente narrador.
12. Narrador genera explicacion (LLM o fallback).
13. API devuelve la narracion.
14. UI muestra todo al usuario.

---

2. Estructura de carpetas

```
eventide/
  src/eventide/
    agents/          Sistema multi-agente
      base.py        Clase base AgenteBase, Mensaje, BuzonGlobal
      buscador.py    Agente de busqueda semantica
      optimizador.py Agente de optimizacion
      narrador.py    Agente de narracion (LLM + fallback)
    api/             API REST
      server.py      Flask app
      routes.py      Endpoints (buscar, agenda, narrar, ciudades, categorias)
      cli.py         Entry point (eventide serve)
```

```
  core/            Modulos core
    embedding.py   Embeddings y busqueda FAISS
    optimizer.py   SA, HC, Random
    knowledge_graph.py Grafo de conocimiento
    simulation.py  Generacion de datos sinteticos
    processing.py  ETL multi-fuente
    text_utils.py  Correccion ortografica
  ui/              Interfaz Streamlit
    app.py         Entry point de la UI
    components.py  Componentes reutilizables
    styles.py      CSS con temas claro y oscuro
  config.py        Configuracion (Pydantic)
  logging_config.py Logging estructurado
tests/             135+ tests
  unit/            Tests unitarios por modulo
  integration/     Tests de integracion (API)
experiments/       Experimentos reproducibles
  01_embedding_comparison/
  02_heuristics_comparison/
docs/              Documentacion
  IA.md
  SIMULACION.md
  SRI.md
  INGENIERIA.md
data/              Datos crudos y procesados
pyproject.toml     Configuracion del proyecto
README.md          README comercial
```

---

3. Patron multi-agente

Cada agente:

- Corre en su propio hilo (thread daemon).
- Tiene una bandeja de entrada (Queue thread-safe).
- Se registra en un BuzonGlobal al inicializarse.
- Recibe mensajes tipados y ejecuta handlers especificos.
- Puede enviar mensajes a otros agentes por nombre.

Ejemplo de comunicacion:

```
self.enviar("buscador", "buscar", {"query": "...", "respuesta_a": "mi_bandeja"})
respuesta = cola_temporal.get(timeout=30)
```

Ventajas:

- Desacoplamiento total entre modulos.
- Escalabilidad horizontal (añadir agentes sin tocar los existentes).
- Resiliencia (un agente que falla no tumba a los demas).

- Testeabilidad (cada agente se testea individualmente).

Coste: latencia extra (aproximadamente 5-10 ms por mensaje).

---

4. Stack tecnico

- Lenguaje: Python 3.11.
- API REST: Flask.
- UI: Streamlit.
- Embeddings: sentence-transformers.
- Busqueda vectorial: faiss-cpu.
- Grafo: networkx.
- LLM: google-generativeai (Gemini).
- Configuracion: pydantic-settings.
- Logging: loguru.
- Testing: pytest.
- Calidad: ruff, black, mypy.

---

5. Como extender el sistema

Añadir un nuevo agente

1. Crear src/eventide/agents/mi_agente.py heredando de AgenteBase.
2. Implementar manejar_mensaje(self, msg).
3. Registrar el agente en cli.py o en el arranque de la API.
4. Añadir tests en tests/unit/test_agents_mi_agente.py.

Añadir un nuevo endpoint

1. Abrir src/eventide/api/routes.py.
2. Añadir @api_bp.route("/mi-endpoint", methods=["POST"]).
3. Usar _enviar_y_esperar para comunicarse con el agente.
4. Añadir tests en tests/integration/test_api.py.

Cambiar el modelo de embeddings

1. Abrir src/eventide/config.py.
2. Cambiar model_name por el nuevo modelo.
3. Regenerar embeddings con python -m eventide.cli seed.

Añadir una nueva fuente de eventos

1. Abrir src/eventide/core/processing.py.
2. Añadir un nuevo handler _process_mi_fuente.
3. Registrar en el EventProcessor.
4. Añadir tests.

---

6. Configuracion y secretos

Toda la configuracion se gestiona con pydantic-settings + archivo .env.

Variables clave:

- PREDICTHQ_API_KEY, SEATGEEK_CLIENT_ID, TICKETMASTER_API_KEY: APIs de eventos.
- GEMINI_API_KEY: LLM para narracion.
- MODEL_NAME: modelo de embeddings (default paraphrase-multilingual-mpnet-base-v2).

Las API keys son SecretStr, asi que no aparecen en logs ni en prints.

---

7. Testing

- 135+ tests unitarios y de integracion.
- Cobertura de todos los modulos.
- Fixtures con scope=module para recursos costosos (modelo SBERT).
- pytest como framework.

Comando:

```
pytest tests/ -v
```

---

8. Reproducibilidad

- Semillas fijas en todos los experimentos (seed=42).
- Datos sinteticos generados con seed fija.
- Modelos descargados de HuggingFace (versionados).
- Resultados guardados en CSV + PNG + conclusions.md.

Modo offline disponible: HF_HUB_OFFLINE=1 para redes sin acceso a HuggingFace.

---

9. Relacion con asignaturas del grado

Eventide integra contenidos de varias asignaturas del grado en Matematica y Computacion:

- Inteligencia Artificial: ver docs/IA.md.
- Simulacion: ver docs/SIMULACION.md.
- Sistemas de Recuperacion de Informacion: ver docs/SRI.md.

Cada documento profundiza en los fundamentos teoricos y experimentales de su area.

---

10. Trabajo futuro

- Migrar a PostgreSQL + pgvector para persistencia.
- Añadir autenticacion JWT en la API.

- Enriquecer el grafo con embeddings de nodos (node2vec).
- Integrar GraphRAG de Microsoft para retrieval aumentado.
- Ampliar a 200+ queries de evaluacion con 3 anotadores (para medir inter-annotator agreement).
- Desplegar en Docker + Kubernetes para produccion.